

ESCUELA COLOMBIANA DE INGENIERÍA PROGRAMACIÓN ORIENTADA A OBJETOS

S06: 2026-01

G03

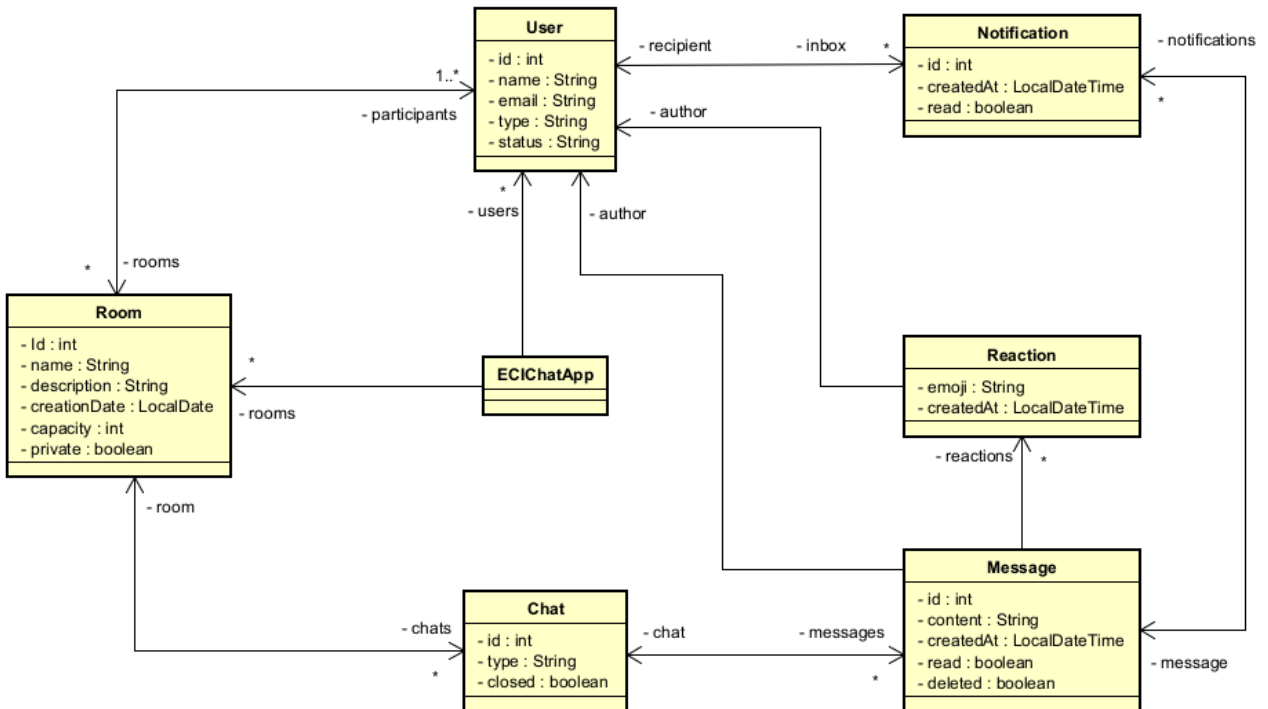
Parcial primer tercio. Nota esperada: ____

ECIChat – Chat académico

La ESCUELA está desarrollando un sistema de mensajería interno para su comunidad académica, llamado ECIChat. Este sistema permitirá que estudiantes y profesores se comuniquen de manera eficiente dentro de la universidad, facilitando la colaboración en proyectos académicos y en grupos de estudio, así como la coordinación de actividades estudiantiles. El personal de la Decanatura de Sistemas ha solicitado a los estudiantes de DOPO de este semestre que construyan la solución que haga posible este proyecto. ECIChat tiene entre sus prioridades las siguientes funciones:

- **Gestión de Usuarios:** Cada usuario en el sistema tiene un tipo de usuario (**type**: puede ser STUDENT o j) y estado (**status**: ACTIVE, INACTIVE, SUSPENDED).
 - o Los usuarios pueden participar en múltiples cuartos simultáneamente, pero no pueden estar registrados más de una vez en el mismo cuarto.
 - o Los usuarios reciben notificaciones en su bandeja personal.
- **Cuartos:** Los usuarios pueden crear cuartos de chat que funcionan como espacios de conversación. Los cuartos pueden ser públicos (cualquiera puede unirse) o privados (sólo por invitación).
 - o Los cuartos solo pueden ser creados por profesores.
 - o Los cuartos, por defecto, siempre tienen el chat GROUP.
 - o Los usuarios pueden unirse a cuartos disponibles y abandonarlos en cualquier momento.
- **Chat:** Dentro de cada cuarto puede haber uno o varios chats donde los participantes intercambian mensajes. El chat puede ser de tipo GROUP (Todos pueden participar), NEWS (solo profesores publican), PROFESSORS (solo profesores pueden escribir y ver mensajes).
 - o Los profesores, si lo desean, pueden agregar los otros tipos de chat al cuarto.
 - o Solo puede haber un chat de cada tipo por cuarto; es decir, el cuarto DOPO-G15 solo puede tener un chat GROUP y uno NEWS. No puede haber dos chats GROUP en el mismo cuarto.
- **Mensajes:** Los usuarios pueden publicar mensajes en los chats de los cuartos a los que pertenezcan, respetando el tipo de chat.
 - o Los mensajes pueden ser eliminados por su autor o por un profesor (exceptuando el chat PROFESSORS).
 - o Cuando un mensaje es eliminado, no se borra físicamente del sistema, sino que se marca como eliminado (soft delete).
 - o Los mensajes eliminados no aparecen en la interfaz de usuarios, pero se mantienen en el sistema por razones de auditoría.
 - o Un mensaje solo se marca como leído cuando todos los miembros del cuarto han leído el mensaje.
- **Reacciones a Mensajes:** Cualquier participante del cuarto puede reaccionar a un mensaje publicado en el chat mediante un emoji.
 - o Cada mensaje puede tener múltiples reacciones de diferentes usuarios.
 - o Un mismo usuario puede dejar solo una reacción por mensaje.
- **Sistema de Notificaciones:** Cuando un usuario publica un mensaje en un chat, el sistema genera automáticamente notificaciones para todos los participantes del cuarto excepto el autor del mensaje.
 - o Las notificaciones se marcan como leídas individualmente cuando cada usuario accede a revisar el mensaje.
 - o El usuario puede marcar todas las notificaciones como leídas si así lo desea.

(Todos los contenedores son HashMap exceptuando las reacciones que son ArrayList)



I. (25%) DIAGRAMA DE MEMORIA

Presente el mapa de memoria correspondiente a:

- ECIChatApp cuenta con múltiples usuarios, entre ellos Jaime Ruiz, profesor de la escuela, y Andrés Ramos, estudiante de la institución.

- Jaime, usuario activo de la aplicación, creó el 10 de enero de este año un cuarto llamado "DOPO G01". Al cuarto solo se pueden unir usuarios con invitación y tiene una capacidad estándar de 24 personas. Al cuarto le creó un chat de tipo *news* con el objetivo de comunicarse de manera más sencilla con sus estudiantes. El 15 de enero envió un mensaje dándoles la bienvenida al curso, el cual a día de hoy todos ya leyeron.
- Andrés, estudiante de la aplicación, se unió al cuarto "DOPO G01" creado por Jaime. Posteriormente, revisó las notificaciones pendientes en su bandeja de entrada con el fin de marcarlas como leídas. Al visualizar el mensaje de bienvenida enviado por el profesor, reaccionó con un "like".

II. (35%) CÓDIGO

MDD

- Estudie el diagrama de secuencia y la especificación (documentación + encabezado) del método
- Actualice el diagrama de clase con los nuevos elementos
- Escriba el código de la clase responsable inicial (encabezado y atributos). Documente el invariante.
- Implemente cada uno de los métodos correspondientes a la solución. Incluya la documentación.

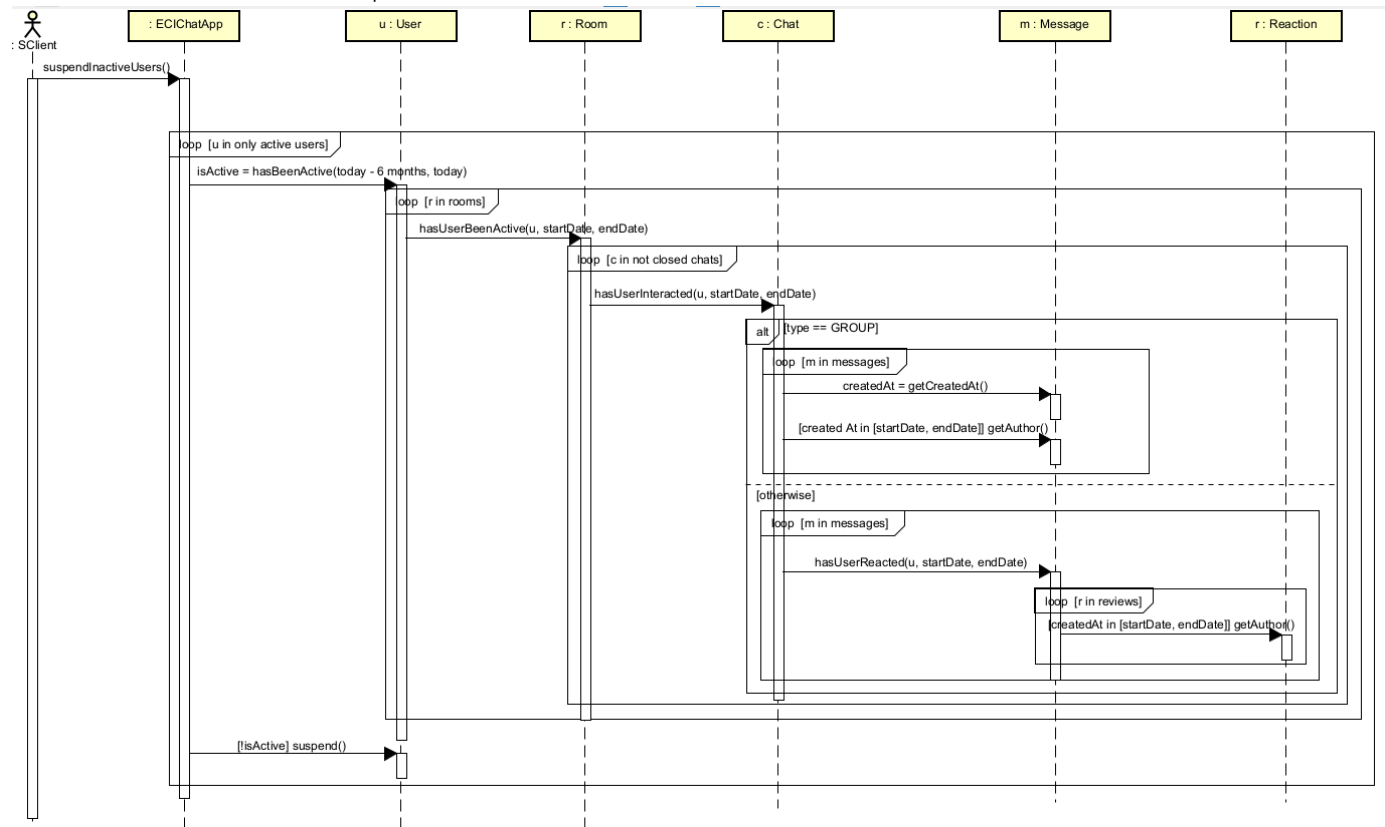
En ECiChatApp

public void suspendInactiveUsers()

Suspende a los usuarios que han permanecido inactivos durante el último semestre.

Reglas:

- Un usuario se considera inactivo si no ha interactuado en ninguno de los cuartos de los que es actualmente integrante.
- Un usuario se considera inactivo en un cuarto cuando no ha realizado ninguna **interacción común** en los chats que se encuentran actualmente abiertos dentro de dicho cuarto. Se entiende por **interacción común** aquella acción que todos los participantes del cuarto pueden realizar según el tipo de chat:
 - En los chats de tipo *GROUP*, todos los usuarios pueden enviar mensajes
 - En los demás tipos de cuartos, la interacción común corresponde a reaccionar a un mensaje.
- Solo se evaluarán los usuarios cuyo estado actual sea **ACTIVE**.
- Solo se verifica sobre los chats que no estan actualmente cerrados



III. (20%) DISEÑO.

Diseñe el método de la clase **User** (incluya especificación formal y diagrama de secuencia decorado) que permita determinar si la mayoría de los cuartos en los que el participa son saludables (*healthy*).

Un cuarto es saludable cuando cumple las siguientes condiciones:

- Al menos 90% de los mensajes de cada chat han sido leídos
- Menos del 20% de los mensajes de cada chat han sido eliminados
- Al menos 50% de los participantes han interactuado en los chats del cuarto desde que este se creó hasta la actualidad

IV. (20%) CONCEPTOS.

- ¿Qué es un ciclo en el desarrollo de software? ¿Cómo deberían definirse?
- ¿Cómo deberían diseñarse clases y métodos para tener una buena distribución de responsabilidades?